

Java Backend Architecture

Interview Series

Chapter 11.9

CRDTs & Collaborative Editing

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 11.9 – CRDTs & Collaborative Editing

Introduction

Conflict-free Replicated Data Types (CRDTs) are mathematical data structures that can be replicated across multiple nodes and merged automatically without conflicts. They achieve this by satisfying three algebraic properties: commutativity ($\text{merge}(A, B) = \text{merge}(B, A)$ — order of merges does not matter), associativity ($\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$ — grouping does not matter), and idempotency ($\text{merge}(A, A) = A$ — merging the same update twice has no additional effect). These properties mean that regardless of network delays, partitions, or reordering, all nodes eventually converge to the same state. CRDTs are the theoretical foundation for many distributed systems that need strong availability without sacrificing convergence.

Real-time collaborative editing (Google Docs-style) is the canonical application of CRDTs. Each user edits their local copy; changes are broadcast and merged using CRDT algorithms. The Replicated Growable Array (RGA) CRDT models a sequence of characters with unique identifiers and logical positions, enabling concurrent inserts and deletes to converge deterministically. Libraries like Yjs and Automerge implement these algorithms with WebSocket integration for real-time synchronization. The competing approach is Operational Transformation (OT), which transforms concurrent operations before applying them — used in Google Docs and Apache Wave, but typically requiring a central server for operation ordering. CRDTs support peer-to-peer architectures while OT generally requires a central coordinator.

CRDT Types – G-Counter, PN-Counter, LWW-Register, OR-Set

CRDT Types Overview – Conflict-Free Replicated Data Types

G-Counter	PN-Counter	LWW-Register	OR-Set
Per-node vector: [N1:3, N2:5] merge = element-wise max value = sum of all	P-vector + N-vector value = $\text{sum}(P) - \text{sum}(N)$ inc P, dec N	(value, timestamp) pair merge = highest timestamp wins simple but clock-skew risk	add-wins: (elem, unique-tag) merge union; remove by tag avoids add/remove race

Required CRDT properties:	Commutativity	Associativity	Idempotency
G-Counter merge example: Node A: [3, 2, 0] Node B: [1, 4, 2] merge = [3, 4, 2] value = $3+4+2 = 9$		OR-Set example: add("a", tag1) remove by tag1 concurrent add("a", tag2) survives remove	

$\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$ [associative] | $\text{merge}(A, B) = \text{merge}(B, A)$ [commutative]

RGA (CRDT) vs Operational Transformation (OT)

RGA (CRDT) vs Operational Transformation (OT) for Text Editing

RGA – Replicated Growable Array

Each char: (id, left-parent-id, value)
Insert at logical position, not index

User A inserts "X" after id:3

User B inserts "Y" after id:3

Both ops applied: "XY" or "YX" by tie-break
(site ID breaks ties deterministically)

OT – Operational Transformation

Transform op B against op A before apply
Requires central server for ordering

A: insert "X" at pos 3 (doc len=5)

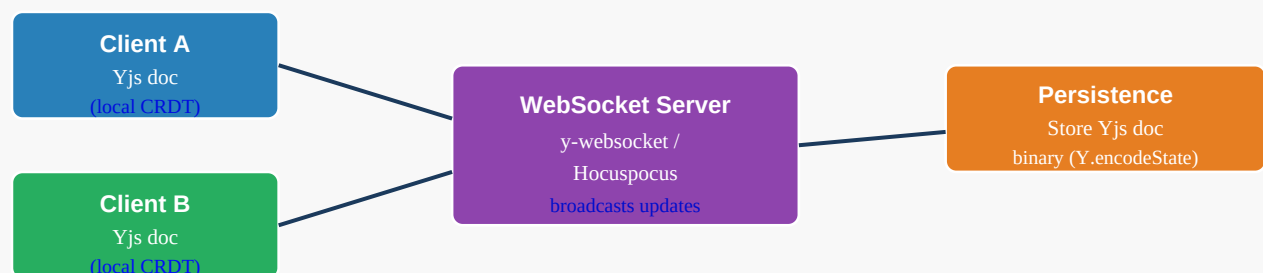
B: insert "Y" at pos 3 (doc len=5)

B transformed: insert "Y" at pos 4
(pos shifted because A already inserted)

Property	RGA / CRDT	OT
P2P support	Yes (no central server needed)	Typically needs central server
Complexity	High (tombstones, IDs)	High (transformation functions)
Libraries	Yjs, Automerge	ShareDB, Google Docs

Real-Time Collaborative Architecture with Yjs + WebSocket

Real-Time Collaborative Editing Architecture



Vector Clock: each node tracks {nodeId: lamportClock} — captures causal happens-before ordering

Causal ordering: if A -> B (A caused B), any node that sees B must have already applied A

Yjs sync: y-protocols sync1/sync2 handshake exchanges missing updates; awareness protocol for cursors/presence

Key Definitions

CRDT	Conflict-free Replicated Data Type: a data structure satisfying commutativity, associativity, and idempotency so all replicas converge automatically
Commutativity	$\text{merge}(A, B) = \text{merge}(B, A)$: the order in which updates are merged does not affect the final state
Associativity	$\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$: grouping of merge operations does not affect the result
Idempotency	$\text{merge}(A, A) = A$: merging the same update multiple times produces the same result as merging it once
G-Counter	Grow-only counter CRDT: per-node vector of increment counts; merge = element-wise max; value = sum of all elements

PN-Counter	Positive-Negative counter: two G-counters P and N; increment adds to P, decrement adds to N; value = sum(P) - sum(N)
LWW-Register	Last-Write-Wins Register: stores (value, timestamp) pair; merge keeps the entry with the highest timestamp; clock skew is a risk
OR-Set	Observed-Remove Set (add-wins): elements are (value, unique-tag) pairs; concurrent add and remove resolved by add winning
RGA	Replicated Growable Array: sequence CRDT for text; each character has a unique ID and left-parent pointer; enables concurrent text editing
Operational Transformation	OT: transforms concurrent operations against each other before applying; requires central server for operation ordering; used in Google Docs
Vector Clock	Per-node logical clock tracking {nodeId: counter}; captures causal happens-before relationships between events
Causal Ordering	If event A causally precedes B (A -> B), any replica that applies B must have already applied A
Yjs	High-performance JavaScript CRDT library; Y.Doc shared document; Y.Text for RGA text; y-websocket for real-time sync; offline-first
Automerge	JavaScript/Rust CRDT library; JSON-like document with automatic conflict resolution; binary protocol; used in local-first apps
State-based vs Op-based CRDT	State-based (CvRDT): merge full state; Op-based (CmRDT): broadcast individual operations; op-based requires at-least-once reliable delivery

CRDT vs OT vs Locking Comparison

Approach	Central Server?	P2P Support	Conflict Resolution	Offline Support	Examples
Pessimistic Locking	Yes	No	Prevents conflicts	No	Traditional CMS
Optimistic + 409	Yes	No	Client resolves manually	No	REST APIs
Operational Transform	Yes (ordering)	Limited	Transform functions	Limited	Google Docs
CRDT (State-based)	Optional	Yes	Automatic (math)	Full	Yjs, Automerge
CRDT (Op-based)	Optional relay	Yes	Automatic	Full	Riak, SoundCloud

Code Examples

1. G-Counter and PN-Counter in Java

```
// G-Counter: grow-only, per-node increment vector
public class GCounter {
    private final String nodeId;
    private final Map<String, Long> counts = new ConcurrentHashMap<>();

    public GCounter(String nodeId) {
        this.nodeId = nodeId;
        counts.put(nodeId, 0L);
    }

    public void increment() {
        counts.merge(nodeId, 1L, Long::sum);
    }

    public long value() {
        return counts.values().stream().mapToLong(Long::longValue).sum();
    }

    // merge = element-wise max (commutativity + associativity + idempotency guaranteed)
    public GCounter merge(GCounter other) {
        GCounter merged = new GCounter(nodeId);
        Set<String> allNodes = new HashSet<>(counts.keySet());
        allNodes.addAll(other.counts.keySet());
        for (String node :
```

```
allNodes) { long a = counts.getOrDefault(node, 0L); long b = other.counts.getOrDefault(node, 0L); merged.counts.put(node, Math.max(a, b)); } return merged; } } // PN-Counter: increment + decrement using two G-Counters public class PNCounter { private final GCounter positive; private final GCounter negative; public void increment() { positive.increment(); } public void decrement() { negative.increment(); } public long value() { return positive.value() - negative.value(); } public PNCounter merge(PNCounter other) { return new PNCounter(positive.merge(other.positive), negative.merge(other.negative)); } } // Use case: distributed like counter, page view counter, shopping cart item count
```

2. LWW-Register and OR-Set

```
// LWW-Register: last write wins by timestamp public class LWWRegister<T> { private T value; private long timestamp; // use HLC (Hybrid Logical Clock) for better accuracy public void set(T newValue, long ts) { if (ts > this.timestamp) { this.value = newValue; this.timestamp = ts; } // If ts == timestamp: deterministic tie-break (e.g., node ID comparison) } public LWWRegister<T> merge(LWWRegister<T> other) { if (other.timestamp > this.timestamp) return other; return this; } } // OR-Set: add-wins set using unique tags public class ORSet<E> { // Map from element to set of unique tags (UUID per add operation) private final Map<E, Set<String>> entries = new HashMap<>(); // Tombstone: removed tags (op-based variant tracks removes) private final Map<E, Set<String>> tombstones = new HashMap<>(); public void add(E element) { String tag = UUID.randomUUID().toString(); entries.computeIfAbsent(element, k -> new HashSet<>()).add(tag); } public void remove(E element) { // Remove ALL current tags for element (marks observed adds as removed) Set<String> tags = entries.getOrDefault(element, Collections.emptySet()); tombstones.computeIfAbsent(element, k -> new HashSet<>()).addAll(tags); } public boolean contains(E element) { Set<String> live = entries.getOrDefault(element, Collections.emptySet()); Set<String> dead = tombstones.getOrDefault(element, Collections.emptySet()); return live.stream().anyMatch(tag -> !dead.contains(tag)); } // merge = union of entries, union of tombstones -> add-wins if concurrent add+remove public ORSet<E> merge(ORSet<E> other) { /* union entries and tombstones */ return this; } }
```

3. RGA – Replicated Growable Array for Collaborative Text

```
// Simplified RGA node public class RGANode { final String id; // unique: nodeId + local sequence (e.g., "A:1") final String leftParentId; // ID of the node this was inserted after (null = head) final char value; boolean deleted; // tombstone: logically deleted but kept for positioning public RGANode(String id, String leftParentId, char value) { this.id = id; this.leftParentId = leftParentId; this.value = value; } } // RGA document: ordered list of nodes public class RGADocument { private final List<RGANode> nodes = new ArrayList<>(); // includes tombstones // Insert character after the node with leftParentId public void insert(String id, String leftParentId, char value) { RGANode newNode = new RGANode(id, leftParentId, value); int insertPos = findInsertPosition(leftParentId, id); nodes.add(insertPos, newNode); } // Concurrent inserts at the same position: tie-break by id (lexicographic) // "A:5" vs "B:3" inserted after same parent -> deterministic order by id private int findInsertPosition(String leftParentId, String newId) { int parentIdx = findNodeIndex(leftParentId); int pos = parentIdx + 1; // Skip nodes that should come before newId (higher id wins tie) while (pos < nodes.size() && nodes.get(pos).leftParentId.equals(leftParentId) && nodes.get(pos).id.compareTo(newId) > 0) { pos++; } return pos; } public void delete(String id) { nodes.stream().filter(n -> n.id.equals(id)) .findFirst().ifPresent(n -> n.deleted = true); } public String getText() { return nodes.stream().filter(n -> !n.deleted) .map(n -> String.valueOf(n.value)) .collect(Collectors.joining()); } } // RGA is the basis for Yjs Y.Text and Automerge text type
```

4. Yjs – Real-Time Collaborative Editing with WebSocket

```
// Client-side (JavaScript/TypeScript with Yjs) import * as Y from 'yjs'; import {
WebsocketProvider } from 'y-websocket'; import { QuillBinding } from 'y-quill'; import Quill
from 'quill'; // Create shared Yjs document const ydoc = new Y.Doc(); // Connect to WebSocket
server (y-websocket or Hocuspocus) const provider = new WebsocketProvider(
'wss://your-server.example.com', // WebSocket URL 'document-room-123', // room name (shared per
document) ydoc ); // Shared text type (RGA under the hood) const ytext =
ydoc.getText('content'); // Bind to Quill editor const quill = new Quill('#editor', { theme:
'snow' }); const binding = new QuillBinding(ytext, quill, provider.awareness); // Awareness:
share cursor position and user info provider.awareness.setLocalStateField('user', { name:
'Alice', color: '#2980B9', cursor: null }); // Offline support: Yjs queues updates while
offline // On reconnect: syncs missing updates automatically (y-protocols sync1/sync2) //
Persist document state (server-side Java/Spring example): // byte[] state =
Y.encodeStateAsUpdate(ydoc); // binary snapshot // byte[] update = Y.encodeStateAsUpdate(ydoc,
remoteState); // delta // Store state bytes in DB (BYTEA column or Redis) // On new client
connect: send stored state + stream live updates
```

5. Vector Clocks for Causal Ordering

```
// Vector clock: {nodeId -> logicalTimestamp} public class VectorClock { private final
Map<String, Long> clock = new HashMap<>(); public void tick(String nodeId) {
clock.merge(nodeId, 1L, Long::sum); } // Merge: element-wise max (same as G-Counter merge)
public VectorClock merge(VectorClock other) { VectorClock merged = new VectorClock();
Set<String> all = new HashSet<>(clock.keySet()); all.addAll(other.clock.keySet()); for (String
node : all) { long a = clock.getOrDefault(node, 0L); long b = other.clock.getOrDefault(node,
0L); merged.clock.put(node, Math.max(a, b)); } return merged; } // Happens-before: this < other
if all entries of this <= other and at least one < public boolean happensBefore(VectorClock
other) { boolean atLeastOneLess = false; for (String node : clock.keySet()) { long a =
clock.get(node); long b = other.clock.getOrDefault(node, 0L); if (a > b) return false; // this
is NOT before other if (a < b) atLeastOneLess = true; } return atLeastOneLess; } // Concurrent:
neither happens before the other public boolean isConcurrentWith(VectorClock other) { return
!this.happensBefore(other) && !other.happensBefore(this); } } // Causal ordering in practice:
// Attach vector clock to each CRDT operation // Before applying op: ensure all causally
preceding ops have been applied // Buffer out-of-order ops until their causal dependencies
arrive
```

6. Automerge – JSON CRDT Document

```
// Automerge: CRDT for JSON-like documents (JavaScript) import * as Automerge from
'@automerge/automerge'; // Create a new document let doc1 = Automerge.init(); doc1 =
Automerge.change(doc1, 'Add task', doc => { doc.tasks = []; doc.tasks.push({ id: 1, title:
'Design API', done: false }); doc.tasks.push({ id: 2, title: 'Write tests', done: false }); });
// Fork: two users edit concurrently let doc2 = Automerge.clone(doc1); // User A completes task
1 doc1 = Automerge.change(doc1, doc => { doc.tasks[0].done = true; }); // User B adds a new
task (concurrent with User A) doc2 = Automerge.change(doc2, doc => { doc.tasks.push({ id: 3,
title: 'Deploy', done: false }); }); // Merge: both changes survive (no conflict – different
fields/positions) let merged = Automerge.merge(doc1, doc2); // merged.tasks = [ // { id: 1,
title: 'Design API', done: true }, // A's change // { id: 2, title: 'Write tests', done: false
}, // { id: 3, title: 'Deploy', done: false } // B's change // ] // Save and load (binary
format, ~10x more compact than JSON) const binary = Automerge.save(merged); const loaded =
Automerge.load(binary); // Java backend: store binary blob, serve as initial doc state to
clients // REST endpoint: GET /documents/{id} -> returns Automerge binary // WebSocket: stream
incremental changes (Automerge.getChanges) in real time
```

Interview Questions & Answers

Q1. What is a CRDT and what three mathematical properties must it satisfy?

A Conflict-free Replicated Data Type (CRDT) is a data structure designed for distributed systems that can be replicated across nodes and merged automatically without any coordination or conflict resolution logic. Three required properties: (1) Commutativity: $\text{merge}(A, B) = \text{merge}(B, A)$ — the order in which updates from different nodes are merged does not affect the final state. (2) Associativity: $\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$ — grouping of merge operations does not matter. (3) Idempotency: $\text{merge}(A, A) = A$ — merging the same update multiple times has no additional effect. These properties guarantee that all replicas eventually converge to the same state regardless of network delays, reordering, or duplicate delivery — without requiring consensus protocols like Paxos or Raft.

Q2. Explain G-Counter and how its merge operation works.

A G-Counter (grow-only counter) maintains a vector of per-node increment counts: { nodeA: 3, nodeB: 5, nodeC: 1 }. Each node can only increment its own slot. The current value is the sum of all slots: $3+5+1=9$. The merge operation takes the element-wise maximum of two counter vectors: $\text{merge}([3,5,1], [4,3,2]) = [4,5,2] \rightarrow \text{value}=11$. This satisfies all three CRDT properties: commutativity (max is symmetric), associativity (max is associative), idempotency ($\text{max}(a,a)=a$). Nodes broadcast their full vector; receivers merge by taking max. This allows all nodes to converge to the same count even with concurrent increments, network delays, or redelivery. Limitation: only increments; PN-Counter extends it with decrements using two G-Counters.

Q3. What is a PN-Counter and how does it support decrements?

A PN-Counter (Positive-Negative Counter) composes two G-Counters: a P counter for increments and an N counter for decrements. To increment: add to P[nodeId]. To decrement: add to N[nodeId]. Current value: $\text{sum}(P) - \text{sum}(N)$. Merge: merge both P and N counters independently using G-Counter merge (element-wise max). Example: Node A increments 5 times ($P=\{A:5\}$), Node B decrements 2 times ($N=\{B:2\}$), concurrent on different replicas. After merge: $P=\{A:5\}$, $N=\{B:2\}$, $\text{value}=5-2=3$. All replicas converge to 3. Use cases: concurrent like/dislike counters, distributed inventory counts. Limitation: the value can be observed as negative during partial replication if decrements propagate faster than increments; handle with application-level bounds if negative values are invalid.

Q4. What is an LWW-Register and what is the risk of using wall-clock timestamps?

An LWW-Register (Last-Write-Wins Register) stores a (value, timestamp) pair. Merge keeps the entry with the highest timestamp. This satisfies CRDT properties when timestamps are globally unique (e.g., using HLC — Hybrid Logical Clocks). Risk: wall-clock timestamps suffer from clock skew — different servers have different system times (typically 100ms-1s with NTP, potentially seconds without). Two concurrent writes from different nodes may have timestamps where the logically later write has a lower timestamp due to skew, causing the wrong value to win. Mitigations: (1) Hybrid Logical Clocks (HLC) — combine physical time with a logical component that always advances monotonically even with skew. (2) Use a distributed sequence generator (Snowflake IDs) instead of wall-clock. (3) Accept clock-skew risk for use cases where occasional incorrect LWW is acceptable (session data, preferences).

Q5. Explain the OR-Set CRDT and why it is called 'add-wins'.

The OR-Set (Observed-Remove Set) represents a set where each element is stored with a set of unique tags (UUIDs generated at add time). To add element 'apple': generate a new UUID tag and store (apple, tag-uuid). To remove 'apple': mark all current tags for 'apple' as tombstones. An element is considered present if it has at least one tag not in the tombstone set. 'Add-wins': if Node A concurrently adds 'apple' (new tag) while Node B removes 'apple' (removes existing tags), after merge: Node A's new tag survives (B didn't see it when it removed), so 'apple' remains present. Add always wins over concurrent remove. This is desirable for shopping carts, collaborative lists, and any case where adding an element concurrently with its removal

should keep it present. The alternative, remove-wins, is less common.

Q6. What is the RGA CRDT and how does it handle concurrent text insertions?

The Replicated Growable Array (RGA) represents a sequence (like text) as a linked list where each element has a unique identifier (nodeId + local sequence, e.g., 'A:5') and a reference to its left neighbor's ID. Insertions are expressed as 'insert character X after element with ID Y'. This is position-independent — the logical position is determined by the left-neighbor reference, not a numeric index. Concurrent insertions at the same position: both reference the same left-neighbor ID. Tie-breaking: elements are sorted by their IDs (e.g., lexicographic order of nodeId:seq). This ensures all nodes produce the same ordering for concurrent inserts. Deletions: logically mark elements as deleted (tombstone) but keep them in the list for positioning. All replicas converge to identical sequences. RGA is the algorithm underlying Yjs Y.Text.

Q7. What is Operational Transformation (OT) and how does it differ from CRDTs?

Operational Transformation transforms concurrent operations before applying them. Example: User A inserts 'X' at position 3. User B deletes character at position 3. When B's delete reaches A (after A's insert): the delete must be transformed to position 4 (because A's insert shifted everything). The transformation function adjusts operation indices based on concurrent operations. OT requires: (1) A central server to establish a canonical ordering of operations. (2) Correct transformation functions for every pair of operation types (insert-insert, insert-delete, etc.). (3) Complex n-party transformation for multiple concurrent users. Differences from CRDTs: OT requires a central server; CRDTs work peer-to-peer. OT transforms operations; CRDTs merge state. OT transformation functions are hard to get right (subtle bugs in multi-user scenarios); CRDTs rely on proven mathematical properties. Modern choice: Yjs/Automerger (CRDT) for new systems; OT in legacy systems (Google Docs original).

Q8. Describe the Yjs architecture for real-time collaborative editing.

Yjs is a high-performance CRDT library. Core components: (1) Y.Doc — the shared document containing typed CRDT structures: Y.Text (RGA-based text), Y.Map (key-value), Y.Array (ordered list). (2) Providers — sync Y.Doc across peers: y-websocket connects to a WebSocket server; y-webrtc for P2P; y-indexeddb for offline persistence in the browser. (3) Awareness — real-time metadata (cursors, user presence) that does NOT need CRDT convergence (ephemeral, last-write-wins per client). Sync protocol: y-protocols implements a two-phase handshake (sync1/sync2) where clients exchange their state vectors (which updates they've seen) and send missing updates. Binary encoding is extremely compact. Server role: the WebSocket server (y-websocket / Hocuspocus) broadcasts updates and optionally persists the document state. It does NOT need to understand the CRDT — just relays binary blobs. Spring Boot backend: receive Yjs binary snapshots, store in DB, serve to new clients.

Q9. What is Automerger and how does it differ from Yjs?

Automerger is a CRDT library (JavaScript and Rust/WASM) that provides a JSON-like document with automatic conflict resolution. Key differences from Yjs: (1) Data model — Automerger models any JSON structure (maps, lists, primitives); Yjs provides specific typed shared types (Y.Text, Y.Map, Y.Array). (2) API style — Automerger uses a functional/immutable API (const newDoc = Automerger.change(doc, fn)); Yjs uses mutable observable objects. (3) Conflict handling — Automerger tracks all concurrent values for the same key (you can inspect them); Yjs uses timestamp-based LWW for map values. (4) Performance — Yjs is generally faster for large text documents; Automerger is more flexible for complex JSON structures. (5) Ecosystem — Yjs has richer editor bindings (Quill, ProseMirror, Monaco, TipTap). Both support offline-first and peer-to-peer. Choose Yjs for collaborative text editing; Automerger for complex shared JSON documents.

Q10. What are vector clocks and how do they establish causal ordering?

A vector clock is a data structure {nodeId: logicalTimestamp} maintained at each node. Rules: (1) Before sending a message, increment your own entry: clock[self]++. (2) On receiving a message: merge by

element-wise max, then increment self. Causal relationships: event A causally precedes event B ($A \rightarrow B$) if A's vector clock is strictly dominated by B's clock (all entries of $A \leq B$, at least one strictly $<$). Concurrent events: neither clock dominates the other. Use in CRDTs: attach vector clocks to operations. Before applying an operation, ensure all causally preceding operations have been applied. Buffer out-of-causal-order operations until their dependencies arrive. This prevents: applying 'delete character at position 5' before the 'insert character at position 5' that it references. Causal ordering is the foundation of consistent collaborative editing behavior.

Q11. What is the difference between state-based CRDTs (CvRDT) and operation-based CRDTs (CmRDT)?

State-based CRDTs (CvRDT — Convergent CRDT): each node broadcasts its entire current state; receivers merge the received state with their local state. The merge function must be commutative, associative, and idempotent. Advantage: simple, works even if messages are dropped or duplicated. Disadvantage: potentially large state to broadcast (e.g., full G-Counter vector). Optimization: send delta states (only changed parts). Operation-based CRDTs (CmRDT — Commutative CRDT): nodes broadcast individual operations; operations must be commutative (order doesn't matter) or the network must guarantee causal ordering. Advantage: small messages (just the operation). Disadvantage: requires reliable causal delivery (at-least-once with causal ordering). Examples: Yjs uses operation-based internally with delta compression; Riak uses state-based (full vector clocks). In practice, modern libraries use hybrid approaches.

Q12. How does Yjs handle offline editing and reconnection?

Yjs is designed for offline-first operation. When offline: all local changes are applied to the local Y.Doc and queued as binary updates. The y-indexeddb provider persists the document state to IndexedDB in the browser, so changes survive page refresh. On reconnect: the y-websocket provider executes the sync1/sync2 protocol: Client sends its state vector (a compact representation of which updates it has seen). Server sends back all updates the client missed. Client sends back all updates the server missed. Both sides apply received updates — Y.Doc merges them using CRDT properties. Because Yjs operations are commutative and idempotent: receiving the same update twice has no effect; receiving updates out of order still produces the correct result (within causal constraints). The result: all clients converge to the same document state after sync, regardless of how long they were offline.

Q13. When would you use a CRDT versus a traditional optimistic concurrency (ETag) approach?

Use CRDTs when: (1) You need real-time multi-user collaboration where all users see each other's changes instantly and any user can edit any part concurrently. (2) You need offline-first support where users can edit without connectivity and sync later. (3) The data type naturally maps to a CRDT (text, counters, sets, JSON). (4) You want automatic conflict resolution without user intervention. Use optimistic concurrency (ETag/If-Match + 409) when: (1) Conflicts are rare and the data is not naturally collaborative (inventory records, financial records). (2) You need full control over conflict resolution logic (business rules). (3) The team is unfamiliar with CRDT complexity. (4) Conflict resolution requires human judgment (showing both values to the user). CRDTs add significant complexity (library integration, data model constraints, testing), so use them only when collaborative editing semantics are a core requirement.

Q14. How do you handle tombstones in CRDTs and what is tombstone accumulation?

Tombstones are logical deletion markers. In RGA: when a character is deleted, it is marked as deleted but kept in the list (because other nodes' insertions may reference it as a left-neighbor). In OR-Set: removed tags accumulate in the tombstone set. Tombstone accumulation problem: over time, tombstones grow unboundedly — a document with 1 million edits accumulates 1 million tombstone entries even if the final document is 1 page. This increases memory, network bandwidth (full state broadcast), and merge time. Garbage collection strategies: (1) Stable tombstone removal — if all nodes have seen a delete (tracked via vector clock), the tombstone can be safely removed. (2) Compaction — periodically create a clean snapshot

of the current state; new nodes join with the snapshot (no need to replay from beginning). (3) Yjs uses an efficient binary encoding and differential updates (deltas) to mitigate the overhead.

Q15. What are the challenges of integrating a CRDT backend with a Java Spring service?

Java Spring CRDT integration challenges: (1) Persistence — CRDT state is binary (Yjs encodes as Uint8Array, Automerge as binary). Store as BYTEA in PostgreSQL or as a blob in S3/object storage. (2) WebSocket server — Spring supports WebSocket via @ServerEndpoint (JSR-356) or Spring WebSocket. The server must relay CRDT updates between clients — effectively a binary message relay, no parsing needed. (3) Access control — the server must validate write permissions before relaying updates. Parse the Yjs update binary (y-protocols format) to check which document is being edited. (4) Persistence service — on each update, save the new document state to DB. Option A: save full state after each update (simple, slightly expensive). Option B: append deltas to an update log; rebuild state on demand. (5) Search indexing — extract text from CRDT document for full-text search. (6) Export — convert Y.Doc to HTML/PDF for download (server-side Node.js or client-side export).

Q16. Explain causal consistency in the context of collaborative editing.

Causal consistency guarantees: if user A makes edit E1 and then makes edit E2 (E2 causally depends on E1 — e.g., E2 is a correction to text inserted by E1), any user who sees E2 must have already seen E1. Without causal ordering: user B might see E2 (a delete operation) before E1 (the insert it deletes) — resulting in a nonsensical state (delete of nothing). Causal consistency implementation in CRDTs: each update carries a vector clock (or dependency set). A node buffers an update until all causally preceding updates have been applied (the update's vector clock is 'dominated' by the node's current clock). Yjs implements this via state vectors — a compact encoding of which updates each node has seen. The sync protocol ensures updates are delivered in causal order. This is weaker than total ordering (no global sequence), but sufficient for collaborative editing where only the author's causal chain matters.

Q17. How would you build a Google Docs-like system using Spring Boot and Yjs?

Architecture: (1) REST API (Spring Boot): CRUD for document metadata (title, owner, collaborators). GET /docs/{id}/state returns the current Yjs binary state (stored in PostgreSQL BYTEA or S3). (2) WebSocket Server (Spring WebSocket or Hocuspocus): handles real-time sync. On connect: send stored Yjs state to client. On update message from client: validate permissions, broadcast to all clients in the room, persist delta to DB. (3) Document storage: persist full Yjs state on every save (or periodically / on disconnect). Also persist deltas for history/undo. (4) Awareness: relay cursor positions and user presence without persistence (ephemeral). (5) Export: server-side Node.js script reads Yjs binary and converts to HTML/DOCX using y-prosemirror + a headless renderer. (6) Search: extract plain text from document and index in Elasticsearch on each save. (7) Access control: verify JWT in WebSocket upgrade handshake; check document sharing permissions.

Q18. What are the performance characteristics of CRDT merge at scale?

CRDT merge performance at scale: (1) G-Counter/PN-Counter merge is $O(n)$ where n is the number of nodes — fast even at thousands of nodes. (2) OR-Set merge is $O(|\text{entries}| + |\text{tombstones}|)$ — grows with the number of add/remove operations over time. (3) RGA merge for text: inserting in a document with M characters is $O(M)$ worst case due to tombstone traversal. Yjs optimizes to $O(k)$ average using balanced tree structures. (4) Full state broadcast: sending complete CRDT state each time is $O(\text{state_size})$. Yjs mitigates this with delta updates (only changed parts) — $O(\text{delta_size})$. (5) Tombstone accumulation increases all operation times over time → periodic compaction needed. Production considerations: measure Yjs document size with real workloads (active collaborative documents often stay under 1MB binary); use Redis pub/sub for WebSocket message broadcasting across multiple Spring instances; use sticky sessions or consistent hashing to route document rooms to the same server.

Q19. What is the difference between strong eventual consistency and strong consistency?

Strong Consistency (linearizability): every read returns the most recently written value; all nodes agree on a single total order of operations. Requires coordination (Paxos, Raft, distributed locks). High latency (round trips for consensus). Unavailable during network partitions (CAP theorem — CP systems). Example: single-master PostgreSQL, etcd, Zookeeper. Strong Eventual Consistency (SEC): all replicas that have received the same set of updates have the same state (convergence), with no coordination required. Reads may return stale data (different replicas at different states). Available during partitions (AP system). Example: Riak with CRDTs, Yjs document. CRDTs guarantee SEC by construction — the mathematical properties ensure convergence. SEC is sufficient for collaborative editing: users tolerate seeing slightly stale cursor positions; the document state always converges. SEC is NOT sufficient for financial transactions or inventory reservations where linearizability is required.

Q20. How do you test a CRDT implementation for correctness?

CRDT testing strategies: (1) Property-based testing (e.g., jqwik in Java, fast-check in JavaScript): generate random sequences of operations on multiple replicas and verify that all replicas converge to the same state after merging. Test that merge is commutative, associative, and idempotent with random inputs. (2) Convergence test: given N replicas each applying a random subset of operations in random order, all should converge to the same result after full sync. (3) Specific conflict scenarios: test concurrent insert at same position (RGA), concurrent add and remove of same element (OR-Set), concurrent increment and decrement (PN-Counter). (4) Regression tests: capture real production conflict scenarios and verify they resolve correctly. (5) Performance tests: benchmark merge time with increasing document size and tombstone count. (6) Network simulation: simulate delayed/dropped/reordered messages and verify convergence. (7) Integration tests: real Yjs clients connected to a real Spring WebSocket server with actual editors.